

Lab 3: Architectural Justification

Food Bank Surplus Redistribution Platform (Problem Statement #62)

Architecture Selection: We chose **Microservices Architecture** for the **Food Bank Surplus Redistribution Platform**.

1. Architectural Choice

The platform is split into small, independently deployable services behind an API Gateway: Identity, Listing, Reservation and Notification. Each service owns its own data and offers a provided interface that other components use through required interfaces, as shown in the component diagram.

2. Two Reasons for This Choice

(a) Load spikes hit only one part of the system. When a restaurant lists a large perishable batch, the Notification Service must alert every shelter within 5 km almost instantly (NFR-001), while the rest of the platform sees normal traffic. With microservices only the Notification Service is scaled out for that spike, and if it fails, donors can still post food and shelters can still reserve batches.

(b) Each business capability has different rules and changes at its own pace. Expiry countdown timers and dietary tags belong to the Listing Service; pickup windows, double-claim prevention and verification codes belong to the Reservation Service; shelter verification belongs to the Identity Service. Separate services let a rule (for example, a new expiry policy) be changed and redeployed without retesting the whole platform, and keep expired food off the claim board.

3. Security Advantage

The API Gateway is the single public entry point: it terminates TLS, validates authentication tokens and rate-limits requests, so internal services are never directly exposed. Only verified shelters may reserve food, so the Reservation Service checks the shelter's status with the Identity Service on every reservation. Because each service has its own datastore, a breach of one service does not expose the data held by the others.

4. Performance Benefit

Services scale independently and communicate asynchronously where speed matters: the Listing Service publishes a new-batch event and responds to the donor immediately, while the Notification Service sends push alerts in the background. Posting a batch is therefore never delayed by notification delivery, which helps meet the instant-alert target under peak load.

5. Why Not the Other Styles

Style	Assessment for this system
Layered	Simple and easy to understand, but all layers are deployed and scaled as one unit, so a notification surge also slows listing and reservation. Tight coupling between layers makes changes risky.
Client-Server	Easy central control and data consistency, but one central server is a single point of failure and a scalability bottleneck, which is risky for food that expires within hours.
Microservices (chosen)	Independent scaling, fault isolation and independent deployment match the bursty, time-critical nature of food rescue.

Trade-off accepted: microservices add operational complexity and network latency. We reduce this with a single API Gateway, containerised deployment and asynchronous messaging.